LECTURE 08 / WEEK 04

MLOps: Machine Learning Operations

Detect impact. Restore service. Prove recovery.

Incident Response for Deployed Models
From monitoring signals to a verified operational decision.



A successful response can still be unusable

Service reliability and prediction usefulness answer different questions.

SERVICE

HTTP 200

The request received a response.

MODEL

Quality unknown

Identity and outcomes still need evidence.

A valid HTTP response may carry the wrong model, wrong semantics or a poor prediction. Availability alone cannot establish predictive quality.

Two independent claims; one does not certify the other.

Use each evidence source for its own question

An aggregate, an event and an external probe have different blind spots.

1	Metrics	Where did behavior change?
2	Events	What happened to this request?
3	External probe	What did the caller observe?

Metrics locate a change. Structured events explain particular requests. External probes see attempts that never reach application logs.

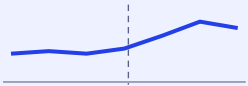
Correlate timestamps, request identities and serving versions.

Read the four signals together

Each signal needs a scope, unit and observation window.

Latency

duration



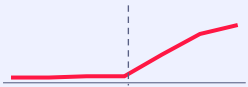
Traffic

requests / second



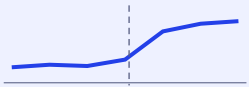
Errors

bad / eligible



Saturation

capacity pressure



Latency describes time; traffic describes demand; errors describe failed outcomes; saturation describes constrained capacity. Interpret them on aligned windows.

Schematic signals; no course-wide threshold is implied.

An SLI is a defined measurement

A reliability fraction is meaningless until its population is explicit.

Good eligible attempts

All eligible attempts

SAME POPULATION / SAME WINDOW

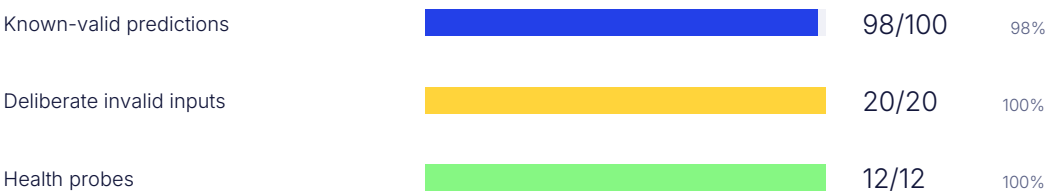
- Endpoint
- Eligibility rule
- Time window

For a chosen endpoint and window, count good eligible attempts over all eligible attempts. State how timeouts, retries and invalid requests enter the calculation.

Zero eligible attempts means no evidence, not 100% success.

The denominator changes the claim

Expected invalid-input rejections are a separate signal.



A known-valid request rejected by a regression is still a failure. An intentional invalid request may correctly receive 4xx; show its rejection rate separately.

Synthetic example: 98/100 inference success differs from 12/12 probe success.

An SLO adds a target and a decision

The measurement, target and evaluation window belong together.

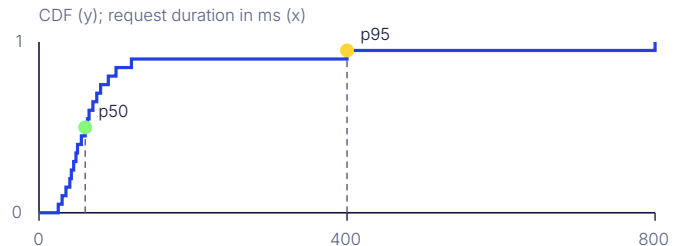
1	Measurement	Eligible population and outcome
2	Objective	Target and evaluation window
3	Policy	Owner, response and review

Choose reliability objectives from user needs and measured capability. A short drill can test an alert or a recovery check; it cannot establish long-term SLO compliance.

No universal latency target or minimum sample count is prescribed.

A latency tail needs enough observations

Report the count and quantile method beside p50, p95 or p99.



n = 20

Nearest-rank quantiles

p50 = 60 ms
p95 = 400 ms

A percentile is a cutoff, not an average. With few requests, a high percentile depends on very few observations. Separate successful-request and failed-request latency.

Synthetic durations; quantile estimates depend on sampling and method.

Identity connects a symptom to a release

Code, runtime image, model and configuration can change independently.

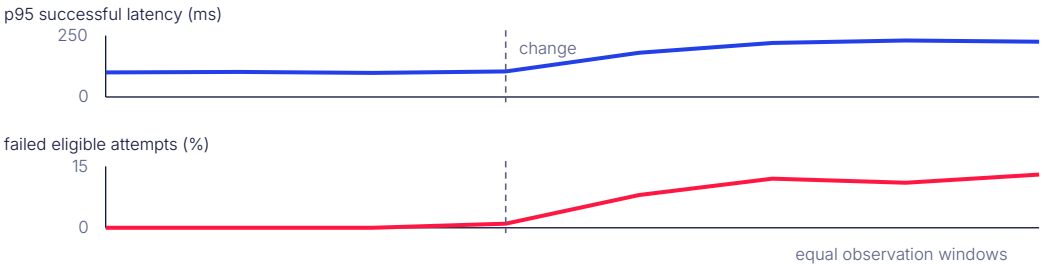
Code	Reviewed commit
Image	Immutable image ID / digest
Model	Version + verified checksum
Configuration	Schema + effective settings

Record what the service actually loaded. Compare it with an independently retained accepted-release record; two endpoints repeating the same string are not independent proof.

A version string identifies a claim; checksum verification links it to bytes.

A release marker narrows the investigation

Time alignment suggests a hypothesis; it does not establish cause.



Overlay changes in code, model or configuration on the same time axis as user-impact signals. Compare both affected and unaffected traffic.

Synthetic series. A coincident change is a lead to investigate.

Observe where meaning can change

A healthy process can consume bad features.

1	Source	Freshness and completeness
2	Features	Schema, transformation, meaning
3	Prediction	Identity and output distribution
4	Outcome	Matched error and coverage

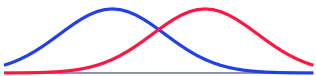
Inspect source freshness, feature validity and transformation, prediction behavior, and eventual outcomes. Each boundary can reveal a different failure.

Range-valid inputs may still have incorrect units or meaning.

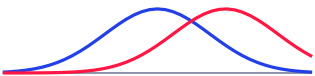
Name what changed before calling it drift

Input, target, prediction and relationship changes are distinct.

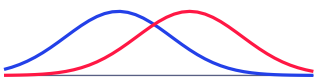
Data: P(X)



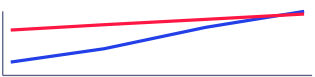
Target: P(Y)



Prediction: P(y_hat)



Concept: P(Y | X)



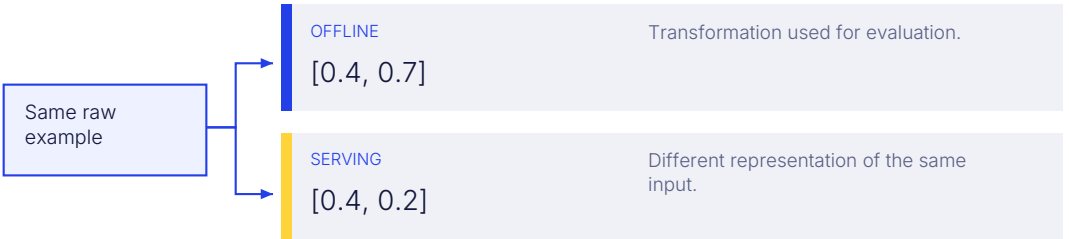
blue: reference red: current schematic distributions / relationships

Data drift changes P(X). Target drift changes P(Y). Prediction drift changes P(y_hat). Concept drift changes P(Y|X). A change in one does not prove all the others changed.

Target drift uses observed outcomes; prediction drift uses model outputs.

Serving skew is a consistency problem

The same raw example should keep the same feature meaning.



Training-serving skew can come from different preprocessing, feature order or defaults. Compare the same input through both paths before blaming a changing world.

Toy example: identical raw input, different transformed values.

Drift is evidence to investigate

It does not establish predictive degradation or prescribe retraining.

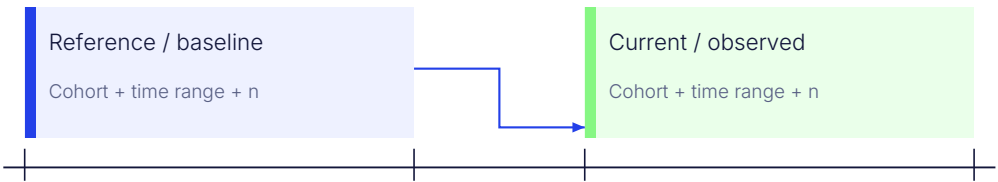


A changed input mix may be benign; quality may deteriorate without a large marginal shift. Establish impact, investigate the mechanism and evaluate any replacement before promotion.

Retraining can help only when its data, cause and evaluated result support it.

Reference windows are part of the definition

Compare like with like, then explain the difference that remains.



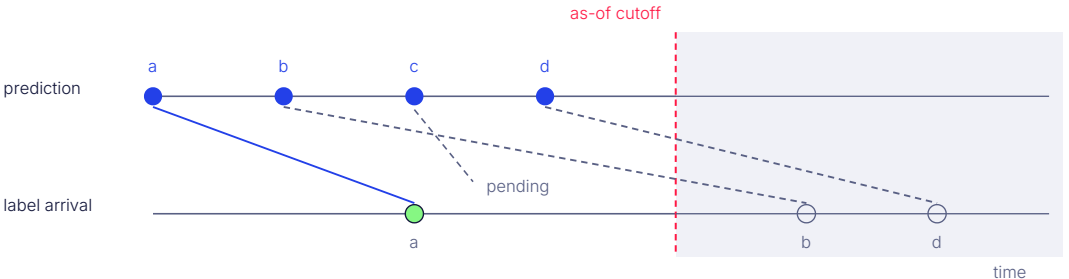
same definitions; explain relevant time and cohort differences

Record feature schema, cohort, time range and sample size for both windows. Energy demand changes with hour, weekday and season; mismatched windows can manufacture a drift alarm.

Time windows are declared before interpretation.

Predictions and labels use different clocks

Event time selects the cohort; label arrival determines what is known.

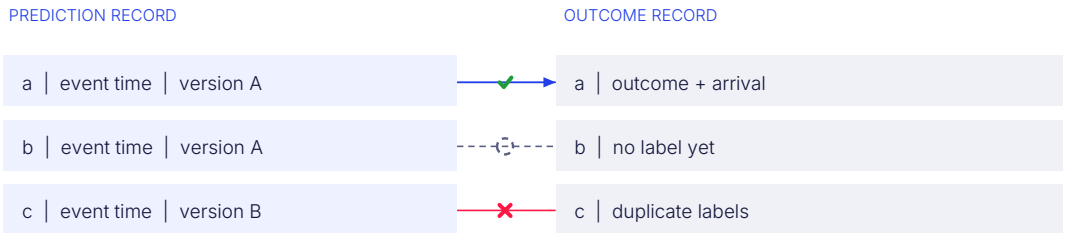


Preserve when a prediction was made, which version made it, and when its outcome became available. Late labels belong to the original prediction cohort.

Missing labels remain unknown; they are never replaced by zero error.

Join at the prediction’s declared grain

A matching label must describe the outcome the prediction intended.



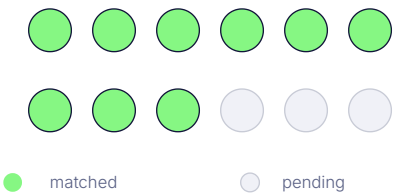
One declared outcome per prediction; do not silently multiply rows.

Keep a stable observation key, event time and served version. Validate uniqueness, matching rules and units; avoid duplicate joins, cross-version mixing and future information.

For repeated requests, distinguish requests from unique scored observations.

A quality number needs a coverage number

State the cohort, served version and label cutoff beside MAE.



9 / 12

75% coverage

3 outcomes pending

Compute MAE only on valid matched pairs, in the target's units. Show matched/eligible observations and pending labels; early labels may be systematically easier or harder cases.

Synthetic cohort: 9 of 12 labels = 75% coverage; three remain pending.

A dashboard supports a decision

Put the signal, its context and the next action in the same view.

POST /predict, window W, known-valid cohort, revision B

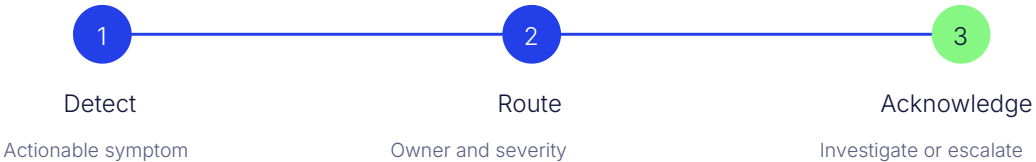
SERVICE IMPACT	LATENCY	MODEL QUALITY
2 / 100 bad	400 ms p95	Unknown
eligible attempts	successful calls: n = 98	0 / 100 labels matched
include failed attempts	method + window stated	owner: investigate change

A useful incident view exposes request population and window, release identities, user impact and label coverage. Detail should answer a responder's next question.

Synthetic dashboard; axes, units, denominators and time scope are explicit.

An alert must reach a responsible person

A dashboard can display alerts; it cannot replace the response route.



Connect an actionable symptom to an owner, severity, acknowledgment and a useful first check. Verify that the notification and recovery states are observable.

A threshold without ownership is an incomplete response policy.

Coordinate the response while restoring service

Uncoordinated changes make the evidence harder to interpret.



Coordinator

Direct the current plan



Operator

Make agreed changes



Recorder

Preserve the timeline



One person directs the current plan, one performs changes, and one records observations and communications. Small teams may combine roles explicitly.

Roles can be combined; responsibilities remain explicit.

Triage establishes scope and what changed

Start with user impact and the last known healthy state.

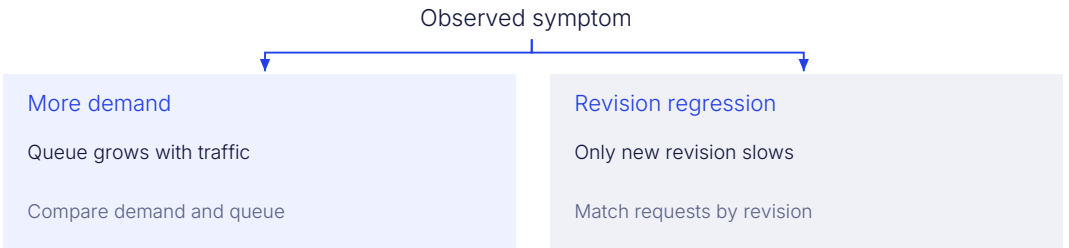
1	Scope	Endpoint, cohort and time
2	Identity	Code, model and configuration
3	Path	Inputs, dependencies and world

Check the service revision, loaded model, configuration, input path and dependencies. Compare affected and unaffected requests; preserve evidence before making a reversible change.

A likely cause remains a hypothesis until evidence supports it.

A hypothesis predicts an observation

Test competing explanations with checks that can distinguish them.

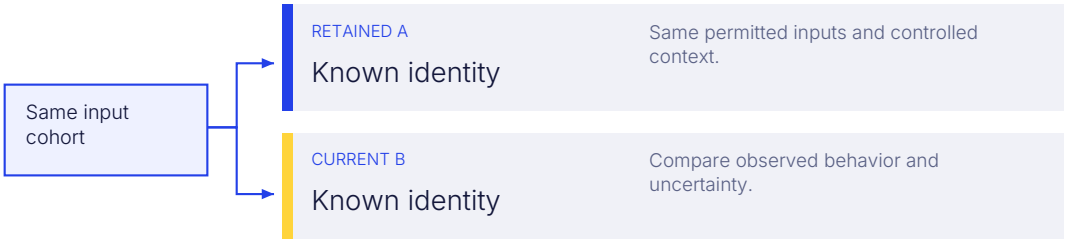


For one symptom, state at least two plausible causes, what each predicts and a discriminating observation. Record support, contradiction and remaining uncertainty.

Illustrative hypotheses, not a diagnosis of a measured service.

Compare versions under matched conditions

Replay supports diagnosis when the comparison is fair.



Run the same permitted inputs through retained versions and preserve identities, configuration and observed results. A shadow comparison does not by itself prove user impact or causal truth.

Control the environment and traffic; do not compare unrelated cohorts.

Choose a mitigation that fits the evidence

Contain impact first; repair the cause through reviewed work.



Contain

Limit the affected path

Mitigate

Restore useful behavior

Correct

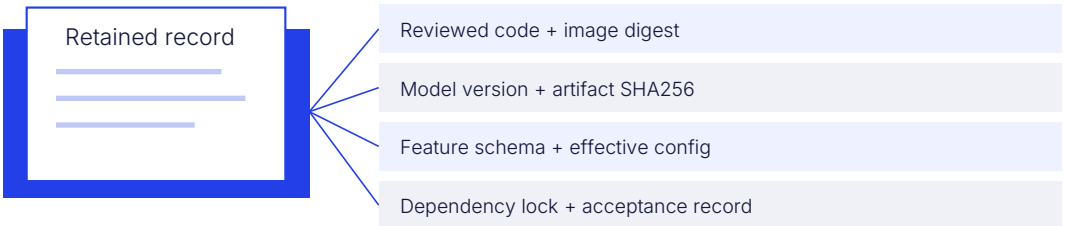
Reviewed preventive repair

Rollback can help a release regression. A guard or approved fallback may contain bad upstream inputs. A later corrective change addresses the mechanism and prevents recurrence.

Rollback, roll-forward and fallback have different preconditions.

Rollback restores a compatible release bundle

The previous model file is only one component.



Retain the accepted model and checksum with compatible code, dependency lock, feature schema and configuration. Name the exact target before changing the running service.

Training a replacement creates a new version; it is not rollback.

Compatibility is a precondition for rollback

Older code can fail against a changed dependency or schema.

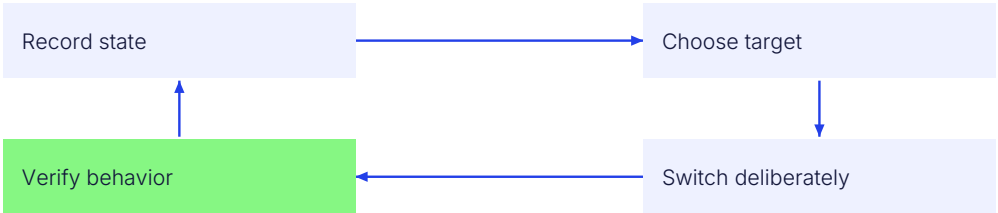
1	Interface	Request and response contract
2	Model path	Feature meaning and dependencies
3	State	External compatibility now

Check the request/response contract, feature semantics, runtime dependencies and external state. If compatibility is unknown, contain the incident and investigate before routing users to an unverified target.

A retained revision is a rollback candidate, not proof of current health.

Treat rollback as a controlled state transition

Preserve evidence, select a target, switch and observe.

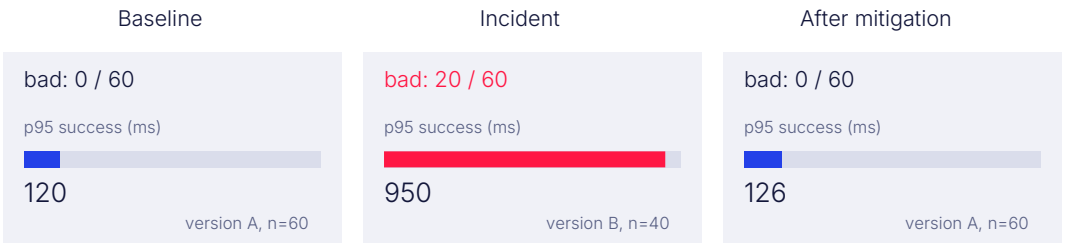


Pause competing changes. Verify the retained target and its compatibility; change the serving selection deliberately. A successful switch starts recovery verification.

Cloud routing changes are not instantaneous; inspect which revisions receive traffic.

Recovery is a comparison of three periods

Use the same signal definitions and a documented observation window.



Compare baseline, incident and fresh post-mitigation traffic. Show sample counts, identities, errors and latency; keep quality conclusions separate when labels are still pending.

Synthetic teaching data; these are not a team’s execution results.

Use independent evidence to verify recovery

Identity, behavior and ongoing signals must support the same conclusion.

1	Identity	Independent expected release record
2	Behavior	Fresh network-level requests
3	Signals	Post-change events and trends

Compare observed identity with a retained expected record. Exercise the service externally with fresh requests. Inspect logs and aggregate signals for the same post-change window.

Repeating one metadata field through three views is one claim repeated.

An ended alert is not enough

Recovery can be partial, temporary or unrelated to the mitigation.

QUIET SIGNAL

Alert stopped

Traffic or telemetry may have disappeared.

VERIFIED RECOVERY

Fresh evidence

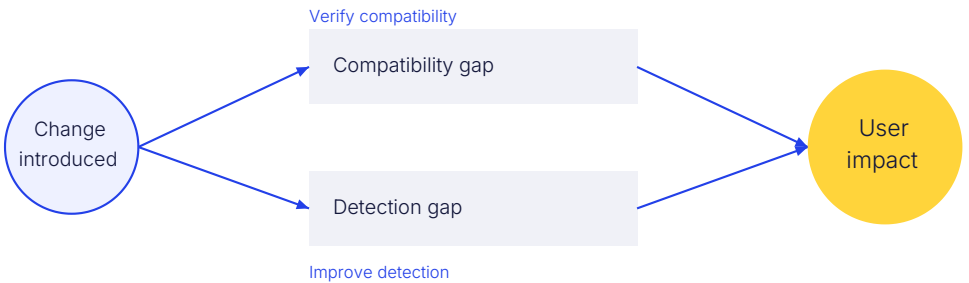
Identity, behavior and observed window agree.

Traffic may have disappeared; labels may still be delayed; the upstream cause may remain. Continue observing for a declared window and state exactly what has recovered.

Rollback need not repair an upstream fault or a changed world.

A postmortem explains the system

Separate trigger, contributing conditions, impact and actions.



Use the timeline and evidence to explain how the incident developed. Record what helped, what delayed recovery and specific preventive actions with owners and verification.

Blameless analysis improves the system without hiding decisions or uncertainty.

A runbook makes the next response faster

It is an executable decision aid with a documented verification path.

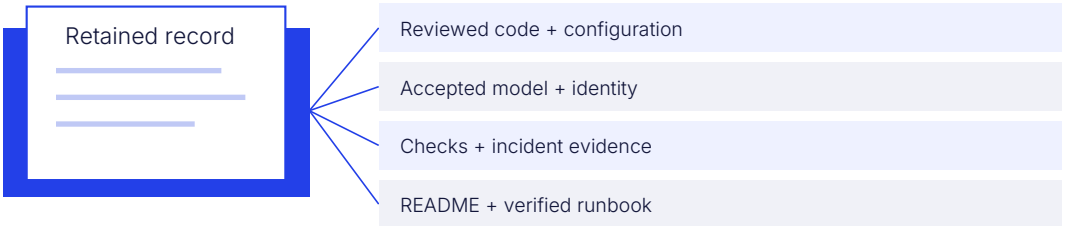


Record recognizable symptoms, first checks, safe actions and rollback prerequisites. Include recovery evidence, escalation and the last verified context.

A runbook remains useful only when its assumptions and steps are checked.

Operational handover preserves the evidence

A release is usable when someone else can identify, run and recover it.

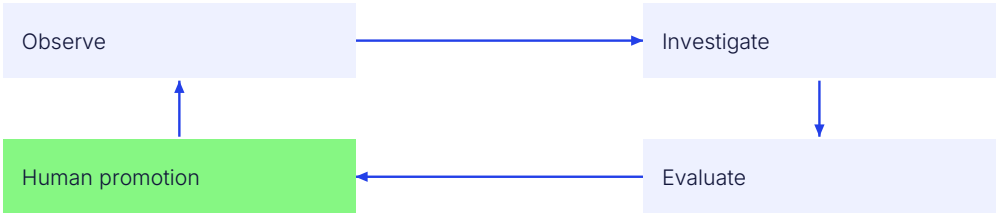


Connect reviewed code, configuration, accepted model and verification records. Explain known limits and recovery procedures so the next owner can assess the service’s state.

A release record links reproducibility with operational responsibility.

Monitoring closes the lifecycle loop

Detection informs a reviewed change; deployment remains a deliberate decision.



Operational evidence can motivate a repair or a new candidate. Evaluation gates preserve fair comparison; an accountable person chooses promotion and verifies the resulting service.

A monitoring alarm does not directly retrain or deploy the model.

LECTURE 08 / ONE ENGINEERING PRINCIPLE

Recovery is a claim.
Fresh evidence makes it credible.

Name the impact. Test the explanation. Restore a compatible state. Verify what users receive.

Instrument the layer that sees the failure

Local telemetry can answer the same operational questions as a hosted dashboard.

1	Application	Request outcome and model identity
2	Runtime	CPU, memory and queue pressure
3	External	Reachability and full request time

Application events describe inference. Runtime metrics describe resource pressure. External requests describe user-visible reachability. A platform dashboard is one implementation choice.

Cloud Run, Prometheus and Grafana are optional implementations.

A health probe checks a declared contract

Liveness and readiness answer different operational questions.

1	Liveness	Process responds
2	Readiness	Usable model loaded
3	Prediction	Inference contract observed

Liveness checks whether a process responds. Readiness checks whether it can serve now. A successful readiness probe does not establish predictive accuracy or every inference path.

Course /health: 200 with a usable model; otherwise 503. Separate routes are optional.

Test the alert path, including recovery

Noise, ownership and missing telemetry affect the response.



Evaluate

Window and missing-data policy

Notify

Owner, severity, escalation

Resolve

Verified recovery state

Define evaluation duration, grouping, severity and escalation. Distinguish a recovered symptom from an absent measurement. The alert route needs an observable acknowledgment and resolution.

Hosted uptime tools are options; the alert contract is tool-agnostic.

Load testing controls the demand

Locust separates virtual-user behavior from the target service.

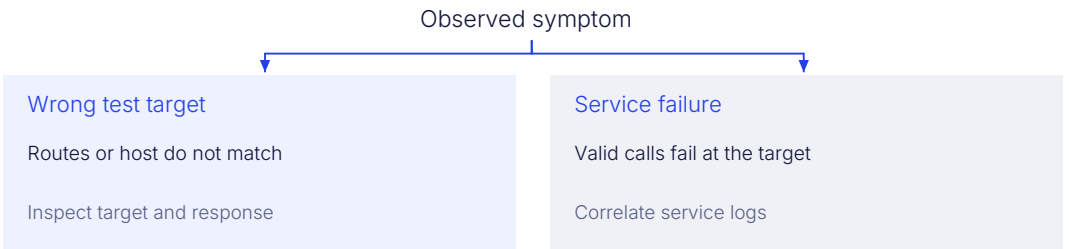


Virtual users execute tasks and wait; their count is not requests per second. The control interface configures a workload, while the target receives the HTTP calls.

Illustrative ports: API 8000; Locust UI 8089. They serve different purposes.

Read the workload before trusting the result

Failures can come from the test setup as well as the service.



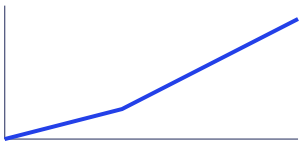
Compare per-endpoint request counts, failures and tail latency with the user ramp. If every request fails, inspect host, routes, payload and response before concluding that capacity is insufficient.

Illustrative failure case; no load test was run for this lecture.

Metric types encode different behavior

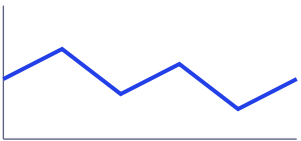
Choose a representation that matches the quantity.

Counter



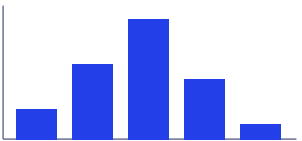
events

Gauge



current state

Histogram



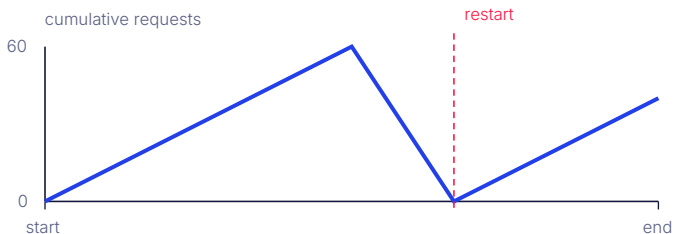
observations

Counters accumulate events and can reset. Gauges show state that rises or falls. Histograms count observed values into buckets; summaries can expose precomputed quantiles.

A count-rate measures event frequency, not average latency.

Counters need reset-aware rates

A restart can reduce the raw counter without undoing past requests.



A reset changes the counter’s origin.

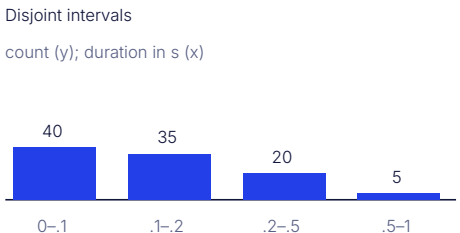
Rate unit:
requests / second

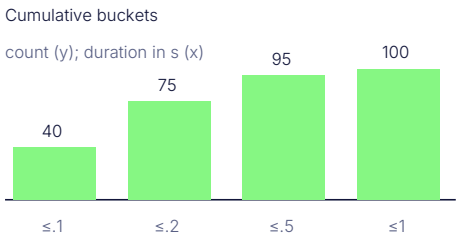
Derive an event rate from counter increase over a stated window. Apply counter-aware rate handling before aggregating series so resets are not mistaken for negative traffic.

Synthetic series. In Prometheus, rate() accounts for counter resets.

Histograms preserve an aggregatable distribution

Classic Prometheus buckets count cumulatively up to each boundary.





A chart may show disjoint intervals while exported buckets are cumulative. Quantiles from histograms are estimates affected by bucket resolution; combine bucket counts before estimating a service quantile.

Synthetic observations in seconds. Do not average per-instance p95 values.

Metric labels must have controlled cardinality

Detailed request identity belongs in events, not every time series.

METRIC SERIES

Bounded labels

Route, status and controlled active versions.

STRUCTURED EVENT

Per-request detail

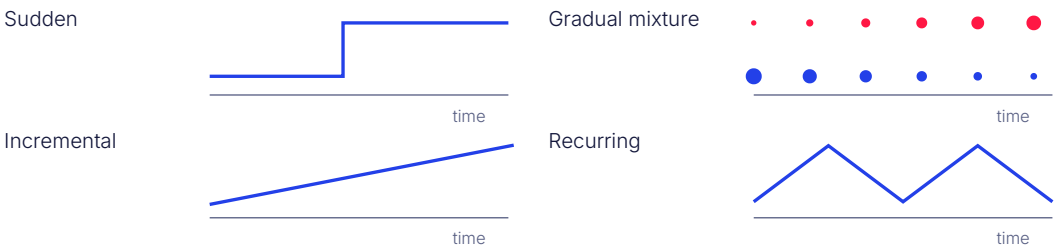
Correlation and selected diagnostic context.

Use bounded route and status categories for aggregation. Each label combination creates a series; unbounded request IDs, user IDs or raw payloads can explode storage.

Version labels need an intentional active-set and retention policy.

Drift can have different temporal shapes

Sudden, gradual, incremental and recurring patterns suggest different checks.

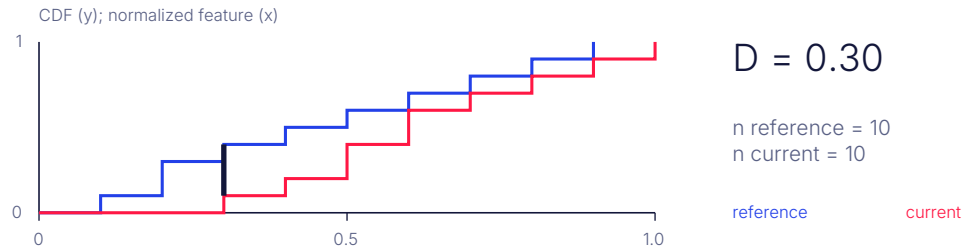


A pattern describes how a distribution changes over time; it does not identify the cause. Seasonality, a release and an upstream fault can need different responses.

Schematic trajectories; no measured drift or cause is asserted.

KS measures the largest empirical CDF gap

The statistic is a distance; its p-value needs assumptions.

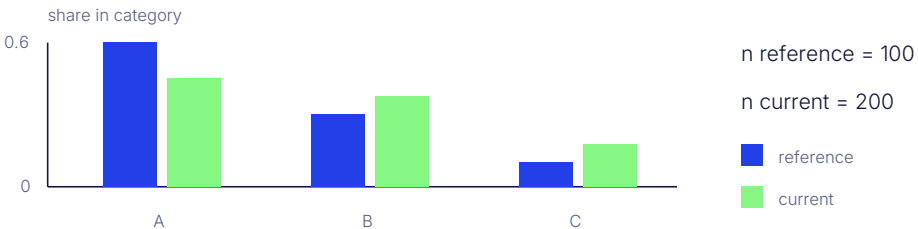


Two-sample KS compares continuous distributions using independent samples. Report sample sizes and effect size; tied, discrete or serially dependent data complicate the standard interpretation.

Synthetic ECDFs. A small p-value is not a model-quality verdict.

Categorical comparisons need aligned categories

Counts, expected frequencies and sample sizes affect the test.



A window-by-category contingency table can compare category mix. Inspect expected counts and sampling assumptions. Distribution methods such as MMD are alternatives when their complexity is justified.

Synthetic counts; unequal sample sizes do not imply drift by themselves.

One application can exhibit several shifts

Taxi-tip examples illustrate definitions, not established causal findings.

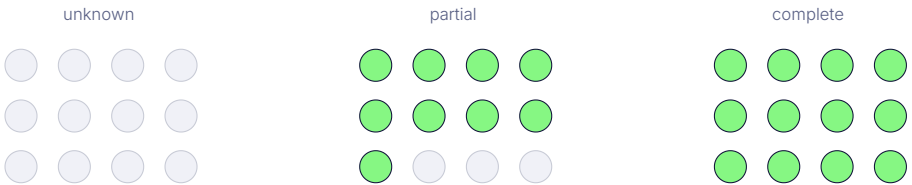
1	Input mix	Different trip locations
2	Target distribution	Different observed tip rate
3	Conditional relationship	Different tipping for similar trips

More trips from one area changes inputs. A different observed tip rate changes targets. Different tipping for otherwise similar trips changes the conditional relationship.

Illustrative hypotheses; no taxi dataset was analyzed.

Early labels can be a biased subset

Coverage describes completeness; it does not establish representativeness.



If one outcome type arrives sooner, a partial quality estimate can differ from the completed cohort. Preserve each as-of snapshot and report which observations are still pending.

Synthetic coverage stages: 0/12, 9/12 and 12/12.

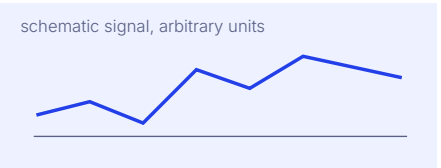
Decision support has an intended human user

An overview and a detail view support different decisions.

Overview: prioritize review

Case 1 context	Case 2 review	Case 3 context
Case 4 context	Case 5 context	Case 6 context

Detail: time and context



Human review remains explicit.

The overview helps prioritize attention. Detail reveals time, units, missingness and context for review. Clinical examples also expose privacy, workflow and alarm-burden constraints.

Abstract interface concept; not a validated clinical tool or medical recommendation.

Precision and recall expose different costs

Define the event, unit of analysis and matching window first.



Recall

TP / (TP + FN)

Precision

TP / (TP + FP)

Recall measures detected actual positives. Precision measures how many positive alerts are true. Threshold choice changes missed events and false alarms; domain context determines acceptable tradeoffs.

Definitions only; no clinical threshold or performance result is claimed.

An explanation is not a causal intervention

Feature attribution describes a model's output under stated assumptions.

MODEL CONTRIBUTION

Describes output

Feature effect within the fitted model.

CAUSAL ACTION

Needs other evidence

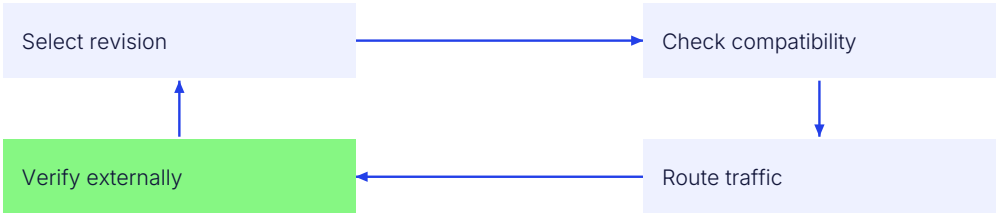
Real-world intervention is a different claim.

A local contribution display can support review of a prediction. It does not prove that changing a feature will cause a better real-world outcome.

Conceptual illustration; no clinical or causal validation is asserted.

Cloud rollback changes traffic routing

A platform example: Cloud Run can target an exact retained revision.

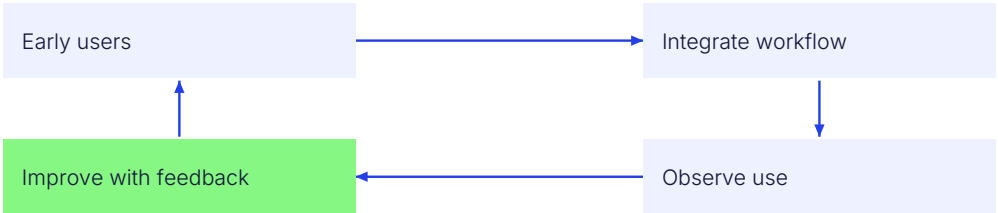


Cloud Run can send traffic to a previous revision. Routing changes are not instantaneous; in-flight requests may still use earlier routing. A switch does not revalidate external dependencies.

Platform example only; no cloud operation was executed.

A service creates value when people use it

Operational handover includes the user's workflow and feedback.



Involve early users, integrate with their existing work and explain intended use and limitations. Assign support and change ownership; monitor adoption alongside technical health.

A reachable API does not establish usefulness, trust or adoption.

Sources: reliability and response

Primary references support the operational concepts and caveats.

- Google SRE: Monitoring Distributed Systems
- SRE Workbook: Implementing SLOs
- Google SRE: Effective Troubleshooting
- Google SRE: Managing Incidents
- Google SRE: Emergency Response
- Google SRE: Postmortem Culture

Official SRE references support measurement, troubleshooting and response.
Use them for concepts, not as a checklist.

Examples in this lecture are conceptual or synthetic.

Sources: measurement and example tools

Read the concepts; the tools are examples, not requirements.

- Prometheus: types, histograms, labels
- Locust: user behavior and tasks
- SciPy: KS and chi-square
- scikit-learn: MAE, precision and recall
- Evidently: distribution drift
- Cloud Run: revision traffic and rollback

Prometheus covers metric semantics; Locust describes controlled users; SciPy and scikit-learn define statistical and quality metrics. Cloud Run documents revision traffic behavior.

Tools are illustrative; no installation is implied.